

16-831: Introduction to Robot Learning, Spring 2024

Policy Gradient Methods (Part II) and Actor-Critic Methods



Generated by GPT-4 with DALLÉ

Instructor: Guanya Shi

Recording Reminder

- ☐ This session will be audio recorded for educational use by other students in this course.

Course Structure

Latest schedule: <https://16-831-s24.github.io/lectures>

- ☐ Topic group 1: Course introduction: What is robot learning and why?
- ☐ Topic group 2: Machine/Deep learning refresher
- ☐ Topic group 3: Imitation learning
- ☒ Topic group 4: Model-free reinforcement learning
- ☐ Topic group 5: Model-based reinforcement learning (one guest lecture)
- ☐ Topic group 6: Reinforcement learning from offline data (one guest lecture)
- ☐ Topic group 7: Bandit, preference-based learning, exploration
- ☐ Topic group 8: Specialized topics (one guest lecture)
- ☐ Project presentation & poster session

Outline

(Some black board derivation today!)

- ❑ Recap: Policy gradient methods
- ❑ Recap: REINFORCE and variance reduction
- ❑ Off-policy policy gradient
- ❑ How to implement PG algorithms
- ❑ Actor-critic algorithms
- ❑ Batch-based actor-critic and off-policy actor-critic

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Outline

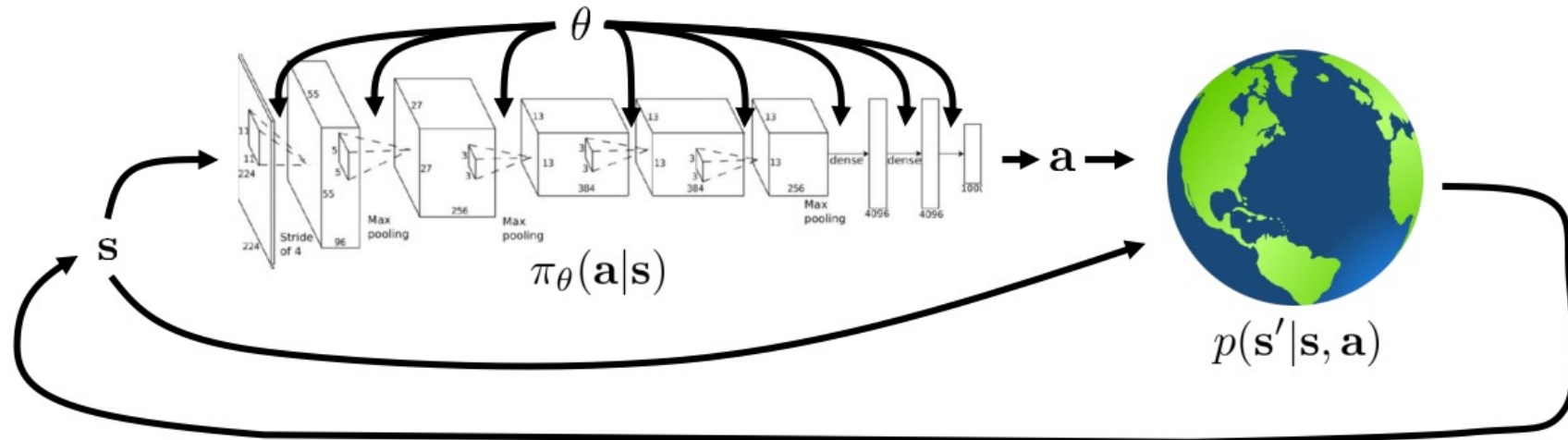
(Some black board derivation today!)

- ❑ Recap: Policy gradient methods
- ❑ Recap: REINFORCE and variance reduction
- ❑ Off-policy policy gradient
- ❑ How to implement PG algorithms
- ❑ Actor-critic algorithms
- ❑ Batch-based actor-critic and off-policy actor-critic

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Policy Gradient (PG)

- ❑ Recap: the goal of RL
- ❑ The goal of policy gradient: directly optimize the policy π_θ via $\nabla_\theta J(\theta)$



$$\underbrace{p_\theta(s_1, \mathbf{a}_1, \dots, s_T, \mathbf{a}_T)}_{p_\theta(\tau)} = p(s_1) \prod_{t=1}^T \pi_\theta(\mathbf{a}_t | s_t) p(s_{t+1} | s_t, \mathbf{a}_t)$$

$J(\theta)$

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(s_t, \mathbf{a}_t) \right]$$

How to Compute $\nabla_{\theta} J(\theta)$?

$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{p_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^T \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\underbrace{\sum_t r(\mathbf{s}_t, \mathbf{a}_t)}_{R(\tau)} \right]$$

- ❑ $R(\tau)$: total reward (return) of the trajectory τ
- ❑ $p_{\theta}(\tau)$: likelihood of the trajectory τ under policy π_{θ}
- ❑ $\nabla_{\theta} J(\theta)$ is hard to compute: θ influences both the action selections and the distribution of states in which those selections are made
- ❑ **Policy gradient theorem:**

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} [R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

REINFORCE Algorithm: Monte Carlo PG

❑ REINFORCE: estimate this expectation by Monte Carlo sampling!

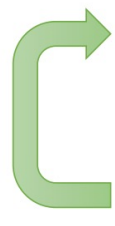
$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} [R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

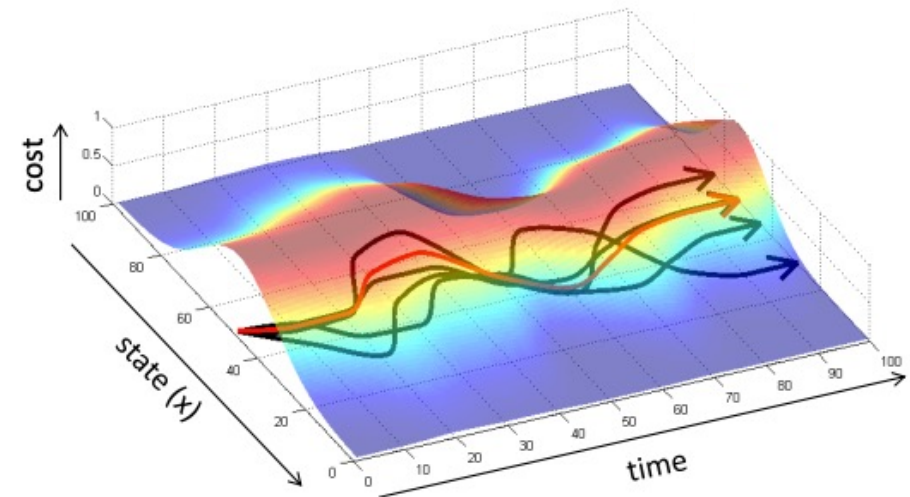
❑ Estimate $\nabla_{\theta} J(\theta)$ using samples $(\tau^i = s_{i,0}, a_{i,0}, s_{i,1}, a_{i,1}, \dots)$:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N R(\tau^i) \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right)$$

❑ REINFORCE algorithm (Sutton et al., NeurIPS 1998):

- On-policy and model-free
- High variance!

- 
1. sample $\{\tau^i\}$ from $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ (run the policy)
 2. $\nabla_{\theta} J(\theta) \approx \sum_i \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^i | \mathbf{s}_t^i) \right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i) \right)$
 3. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$



Reducing Variance

□ Causality: policy at time t' cannot affect reward at $t < t'$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_t \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \left(\underbrace{\sum_{t'=t}^{\infty} r(s_{i,t'}, a_{i,t'})}_{\text{Reward to go!}} \right)$$

Its expectation is $Q^{\pi}(s, a)$

$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [Q^{\pi}(s, a) \nabla_{\theta} \log \pi_{\theta}(a | s)]$$

□ Baselines: adding a bias term $b(s)$ doesn't influence the gradient!

$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [(Q^{\pi}(s, a) - b(s)) \nabla_{\theta} \log \pi_{\theta}(a | s)]$$

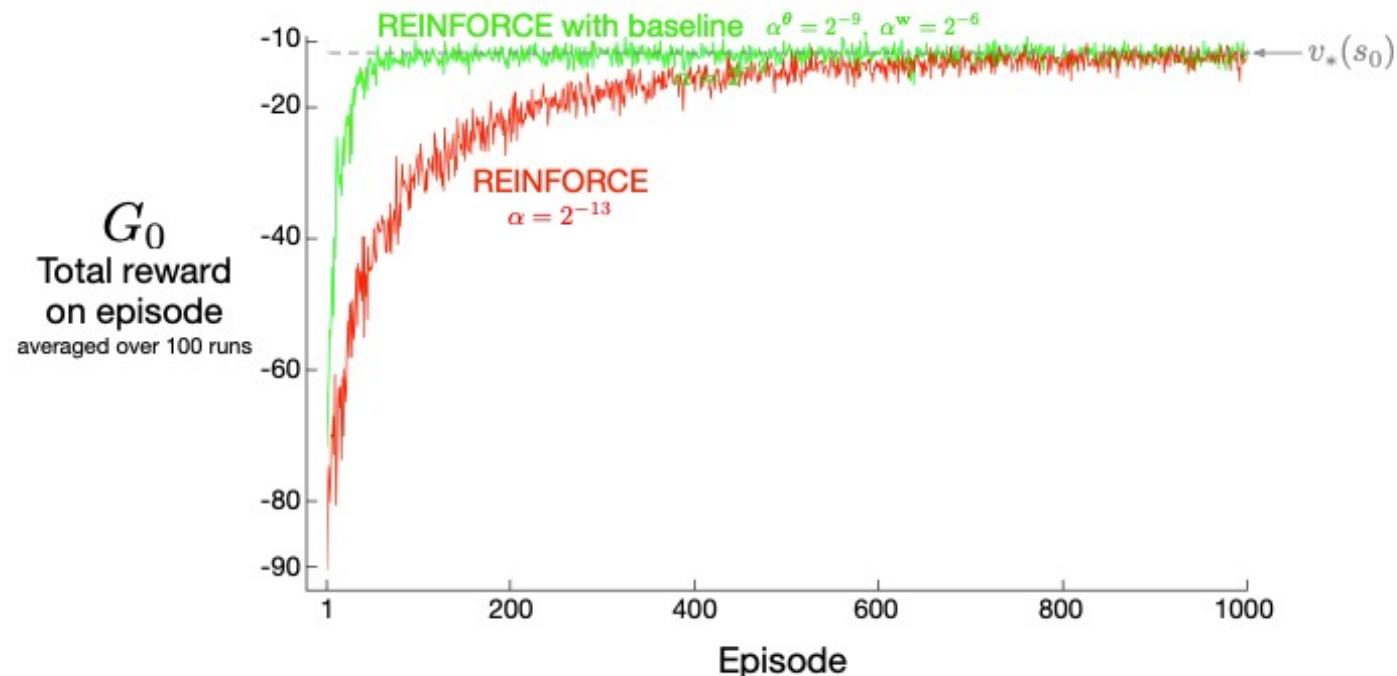
Reducing Variance

□ Baselines: adding a bias term $b(s)$ doesn't influence the gradient!

$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [(Q^{\pi}(s, a) - b(s)) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

□ Common choice: some “smart” constant or $b(s) \approx V^{\pi}(s)$

- In this case, $Q^{\pi}(s, a) - V^{\pi}(s) = A^{\pi}(s, a)$ is the **advantage** function!



Outline

(Some black board derivation today!)

- ☐ Recap: Policy gradient methods
- ☐ Recap: REINFORCE and variance reduction
- ☒ Off-policy policy gradient
- ☐ How to implement PG algorithms
- ☐ Actor-critic algorithms
- ☐ Batch-based actor-critic and off-policy actor-critic

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Off-policy Policy Gradient: Why?

□ Policy gradient theorem:

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} [R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [(Q^{\pi}(s, a) - b(s)) \nabla_{\theta} \log \pi_{\theta}(a | s)]$$

□ Why is PG **on-policy**? The red term $(\tau \sim p_{\theta}(\tau), s \sim p_{\theta}, a \sim \pi_{\theta})$ is trouble!

□ Why is it bad?

- The neural network π_{θ} change only a little bit with each gradient step
- On-policy learning can be extremely inefficient (cannot reuse old data)!

can't just skip this! 



1. sample $\{\tau^i\}$ from $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ (run the policy)
2. $\nabla_{\theta} J(\theta) \approx \sum_i (\sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^i | \mathbf{s}_t^i)) (\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i))$
3. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

□ What if we have samples from other policies?

Off-policy Policy Gradient

- ❑ Let's compute $\nabla_{\theta'} J(\theta')$ using samples from π_{θ}
- ❑ Step 1: importance sampling

$$J(\theta') = E_{\tau \sim p_{\theta}(\tau)} \left[\frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} R(\tau) \right]$$

- ❑ Step 2: compute the gradient and use the identity

$$\nabla_{\theta'} J(\theta') = E_{\tau \sim p_{\theta}(\tau)} \left[\frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} \nabla_{\theta'} \log p_{\theta'}(\tau) R(\tau) \right]$$

- ❑ Step 3:

$$\nabla_{\theta'} J(\theta') = E_{\tau \sim p_{\theta}(\tau)} \left[\frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} \left(\sum_{t=1} \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) \right) \left(\sum_{t=1} r(s_t, a_t) \right) \right]$$

importance sampling

$$\begin{aligned} E_{x \sim p(x)}[f(x)] &= \int p(x) f(x) dx \\ &= \int \frac{q(x)}{q(x)} p(x) f(x) dx \\ &= \int q(x) \frac{p(x)}{q(x)} f(x) dx \\ &= E_{x \sim q(x)} \left[\frac{p(x)}{q(x)} f(x) \right] \end{aligned}$$

a convenient identity

$$p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) = \nabla_{\theta} p_{\theta}(\tau)$$

Off-policy Policy Gradient

□ Step 4:

$$\begin{aligned}\nabla_{\theta'} J(\theta') &= E_{\tau \sim p_{\theta}(\tau)} \left[\frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} \left(\sum_{t=1} \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) \right) \left(\sum_{t=1} r(s_t, a_t) \right) \right] \\ &= E_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1} \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) \left(\prod_{t=1} \frac{\pi_{\theta'}(a_t | s_t)}{\pi_{\theta}(a_t | s_t)} \right) \left(\sum_{t=1} r(s_t, a_t) \right) \right]\end{aligned}$$

□ Step 5: causality

$$= E_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1} \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) \left(\prod_{t=1} \frac{\pi_{\theta'}(a_t | s_t)}{\pi_{\theta}(a_t | s_t)} \right) \underbrace{\left(\sum_{t'=t} r(s_{t'}, a_{t'}) \right)}_{\text{reward to go!}} \right]$$

□ Problem: green term exponential in horizon, inefficient to compute (also high variance)!

reward to go!
 $\sim Q^{\pi}(s, a)$

Off-policy Policy Gradient: Approximation

□ Let's compute $\nabla_{\theta'} J(\theta')$ using samples from $\pi_{\theta}(a|s)$ (**behavior policy**)

$$\nabla_{\theta'} J(\theta') \propto E_{(s,a) \sim \pi_{\theta'}} [Q^{\pi_{\theta'}}(s, a) \nabla_{\theta'} \log \pi_{\theta'}(a|s)]$$

□ Step 1: Importance sampling. Recap notations:

- $p_{\theta}(s, a)$: state/action joint distribution under π_{θ}
- $p_{\theta}(s)$: state distribution under π_{θ}

$$\begin{aligned} \nabla_{\theta'} J(\theta') &\propto E_{(s,a) \sim \pi_{\theta}} \left[\frac{p_{\theta'}(s, a)}{p_{\theta}(s, a)} Q^{\pi_{\theta'}}(s, a) \nabla_{\theta'} \log \pi_{\theta'}(a|s) \right] \\ &= E_{(s,a) \sim \pi_{\theta}} \left[\frac{\textcolor{red}{p_{\theta'}(s)}}{\textcolor{red}{p_{\theta}(s)}} \frac{\pi_{\theta'}(a|s)}{\pi_{\theta}(a|s)} Q^{\pi_{\theta'}}(s, a) \nabla_{\theta'} \log \pi_{\theta'}(a|s) \right] \end{aligned}$$

□ Step 2: Ignore the red part (we will explain why it is reasonable later). Approximation:

$$\nabla_{\theta'} J(\theta') \approx E_{(s,a) \sim \pi_{\theta}} \left[\frac{\pi_{\theta'}(a|s)}{\pi_{\theta}(a|s)} Q^{\pi_{\theta'}}(s, a) \nabla_{\theta'} \log \pi_{\theta'}(a|s) \right]$$

Outline

(Some black board derivation today!)

- ☐ Recap: Policy gradient methods
- ☐ Recap: REINFORCE and variance reduction
- ☐ Off-policy policy gradient
- ☒ How to implement PG algorithms
- ☐ Actor-critic algorithms
- ☐ Batch-based actor-critic and off-policy actor-critic

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Implementing PG Algorithms

□ Expectation form:

$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [Q^{\pi}(s, a) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

□ Q^{π} is unknown, so estimating using samples ($\sum_{t'=t}^{\infty} r(s_{t'}, a_{t'}) = \hat{Q}^{\pi}(s_t, a_t) \approx Q^{\pi}(s, a)$):

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_t \hat{Q}_{i,t}^{\pi} \nabla_{\theta} \log \pi_{\theta}(a_{i,t}|s_{i,t})$$

□ You don't need manually compute $\sum_t \nabla_{\theta} \log \pi_{\theta}(a_{i,t}|s_{i,t})$! Using automatic diff!

- Define a “pseudo” loss $L(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_t \hat{Q}_{i,t}^{\pi} \cdot \log \pi_{\theta}(a_{i,t}|s_{i,t})$
- Then do automatic diff!

□ Example: Gaussian policies

$$\text{example: } \pi_{\theta}(\mathbf{a}_t|\mathbf{s}_t) = \mathcal{N}(f_{\text{neural network}}(\mathbf{s}_t); \Sigma)$$

$$\log \pi_{\theta}(\mathbf{a}_t|\mathbf{s}_t) = -\frac{1}{2} \|f(\mathbf{s}_t) - \mathbf{a}_t\|_{\Sigma}^2 + \text{const}$$

$$\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t|\mathbf{s}_t) = -\frac{1}{2} \Sigma^{-1} (f(\mathbf{s}_t) - \mathbf{a}_t) \frac{df}{d\theta}$$

Implementing PG Algorithms

❑ Discount factors (details in the HW assignment): Very similar!

The first way applies the discount on the rewards from full trajectory:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_{it} | s_{it}) \right) \left(\sum_{t'=0}^{T-1} \gamma^{t'} r(s_{it'}, a_{it'}) \right) \quad (9)$$

and the second way applies the discount on the “reward-to-go:”

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_{it} | s_{it}) \left(\sum_{t'=t}^{T-1} \gamma^{t'-t} r(s_{it'}, a_{it'}) \right). \quad (10)$$

❑ Baselines. This holds for any $b(s)$:

$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [(Q^{\pi}(s, a) - b(s)) \nabla_{\theta} \log \pi_{\theta}(a | s)]$$

❑ Baselines in practice (GAE, **Generalize Advantage Estimation**):

- Estimate the advantage function $Q^{\pi}(s, a) - V_{\phi}^{\pi}(s)$
- Monte Carlo for $Q^{\pi}(s, a)$, another value network for $V_{\phi}^{\pi}(s)$
- Actor-Critic (more in later slides)!

$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [(Q^{\pi}(s, a) - V_{\phi}^{\pi}(s)) \nabla_{\theta} \log \pi_{\theta}(a | s)]$$

Outline

(Some black board derivation today!)

- ☐ Recap: Policy gradient methods
- ☐ Recap: REINFORCE and variance reduction
- ☐ Off-policy policy gradient
- ☐ How to implement PG algorithms
- ☒ Actor-critic algorithms
- ☐ Batch-based actor-critic and off-policy actor-critic

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Actor-Critic Methods

□ Recap: policy gradient with baselines

$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [(Q^{\pi}(s, a) - b(s)) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

□ In practice (GAE, **Generalize Advantage Estimation**):

- Estimate the advantage function $A^{\pi}(s, a) = Q^{\pi}(s, a) - V_{\phi}^{\pi}(s)$
- Example: Monte Carlo for $Q^{\pi}(s, a)$, another value network for $V_{\phi}^{\pi}(s)$

$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [(Q^{\pi}(s, a) - V_{\phi}^{\pi}(s)) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

□ **Actor-critic methods**: learning **both** policies (**actor**) and value functions (**critic**)

□ How to learn $V_{\phi}^{\pi}(s)$ (or directly $A^{\pi}(s, a)$)?

Actor-Critic Methods

□ Recap: policy gradient with baselines

$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [(Q^{\pi}(s, a) - b(s)) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

□ In practice (GAE, **Generalize Advantage Estimation**):

- Estimate the advantage function $A^{\pi}(s, a) = Q^{\pi}(s, a) - V_{\phi}^{\pi}(s)$
- Example: Monte Carlo for $Q^{\pi}(s, a)$, another value network for $V_{\phi}^{\pi}(s)$

$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [(Q^{\pi}(s, a) - V_{\phi}^{\pi}(s)) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

□ Option 1: Monte Carlo evaluation for $V_{\phi}^{\pi}(s)$

$$\text{training data: } \left\{ \left(\mathbf{s}_{i,t}, \underbrace{\sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'})}_{y_{i,t}} \right) \right\}$$

$$\text{supervised regression: } \mathcal{L}(\phi) = \frac{1}{2} \sum_i \left\| \hat{V}_{\phi}^{\pi}(\mathbf{s}_i) - y_i \right\|^2$$



Actor-Critic Methods

- ❑ Can we do better?
- ❑ Recap: Estimating $V_{\phi}^{\pi}(s)$ is exactly a policy evaluation problem (lecture 7)!
 - Bellman equation: $V^{\pi}(s) = E_{\pi}[r(s, a) + \gamma V^{\pi}(s')]$
- ❑ Idea: compute the target $y = r(s, a) + \gamma V_{\phi}^{\pi}(s')$
 - Then use the loss $L(\phi) = \frac{1}{2} (y - V_{\phi}^{\pi}(s))^2$ to update V_{ϕ}^{π}
 - y is often referred to as a “**bootstrapped**” estimate
- ❑ Difference from Q-learning: there is no max operator! So more “stable”
 - Q-learning aims to approximate Q^*
 - V_{ϕ}^{π} is **policy conditioned**

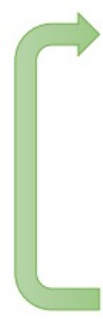
Actor-Critic Algorithms

□ Actor-critic aims to estimate the red term:

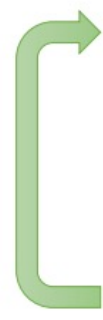
$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [(Q^{\pi}(s, a) - V^{\pi}(s)) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

$$Q^{\pi}(s, a) - V^{\pi}(s) \approx r(s, a) + \gamma V_{\phi}^{\pi}(s') - V_{\phi}^{\pi}(s)$$

batch actor-critic algorithm:

- 
1. sample $\{\mathbf{s}_i, \mathbf{a}_i\}$ from $\pi_{\theta}(\mathbf{a}|\mathbf{s})$ (run it on the robot)
 2. fit $\hat{V}_{\phi}^{\pi}(\mathbf{s})$ to sampled reward sums
 3. evaluate $\hat{A}^{\pi}(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}'_i) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_i)$
 4. $\nabla_{\theta} J(\theta) \approx \sum_i \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^{\pi}(\mathbf{s}_i, \mathbf{a}_i)$
 5. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

online actor-critic algorithm:

- 
1. take action $\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$
 2. update $\hat{V}_{\phi}^{\pi}$ using target $r + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}')$
 3. evaluate $\hat{A}^{\pi}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}') - \hat{V}_{\phi}^{\pi}(\mathbf{s})$
 4. $\nabla_{\theta} J(\theta) \approx \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}|\mathbf{s}) \hat{A}^{\pi}(\mathbf{s}, \mathbf{a})$
 5. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

Actor-Critic Algorithms

❑ Question: Is actor-critic **on-policy** or **off-policy**?

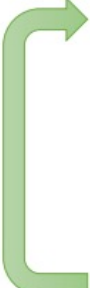
❑ Question: What if V_ϕ^π is a bad estimation of $V^\pi(s)$?

$$\nabla_\theta J(\theta) \propto E_{s \sim p_\theta, a \sim \pi_\theta} [(Q^\pi(s, a) - V^\pi(s)) \nabla_\theta \log \pi_\theta(a|s)]$$

batch actor-critic algorithm:

- 
1. sample $\{\mathbf{s}_i, \mathbf{a}_i\}$ from $\pi_\theta(\mathbf{a}|\mathbf{s})$ (run it on the robot)
 2. fit $\hat{V}_\phi^\pi(\mathbf{s})$ to sampled reward sums
 3. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
 4. $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
 5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

online actor-critic algorithm:

- 
1. take action $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$
 2. update $\hat{V}_\phi^\pi$ using target $r + \gamma \hat{V}_\phi^\pi(\mathbf{s}')$
 3. evaluate $\hat{A}^\pi(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}') - \hat{V}_\phi^\pi(\mathbf{s})$
 4. $\nabla_\theta J(\theta) \approx \nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s}) \hat{A}^\pi(\mathbf{s}, \mathbf{a})$
 5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

Actor-Critic Algorithms

❑ Question: Is actor-critic **on-policy** or **off-policy**?

- Standard actor-critic is on-policy!
- Data has to come from the current policy to compute the correct gradient!

❑ Question: What if V_{ϕ}^{π} is a **bad** estimation of $V^{\pi}(s)$?

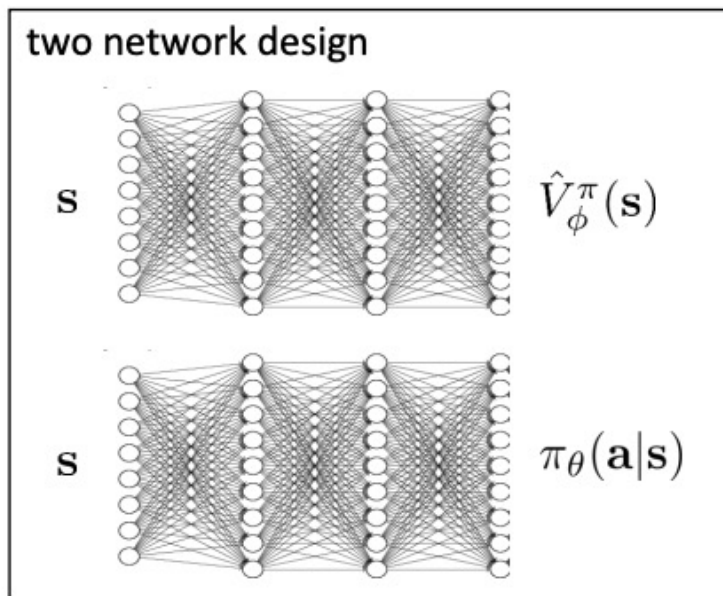
- Depends on how we use V_{ϕ}^{π} ...

$\hat{A}_C^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+1}) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_t)$	+ lower variance - higher bias if value is wrong (it always is)
$\hat{A}_{MC}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{\infty} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_t)$	+ no bias - higher variance (because single-sample estimate)

- Another option: n-step return (using eligibility trace)!

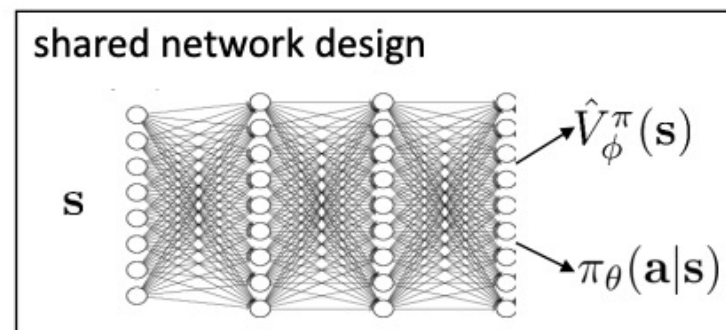
$$\hat{A}_n^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_t) + \gamma^n \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+n})$$

Architecture Design in Actor-Critic



+ simple & stable

- no shared features between actor & critic



online actor-critic algorithm:

1. take action $\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$
2. update $\hat{V}_{\phi}^{\pi}$ using target $r + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}')$
3. evaluate $\hat{A}^{\pi}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}') - \hat{V}_{\phi}^{\pi}(\mathbf{s})$
4. $\nabla_{\theta} J(\theta) \approx \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}|\mathbf{s}) \hat{A}^{\pi}(\mathbf{s}, \mathbf{a})$
5. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

Outline

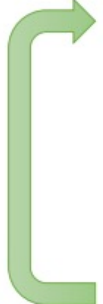
(Some black board derivation today!)

- ☐ Recap: Policy gradient methods
- ☐ Recap: REINFORCE and variance reduction
- ☐ Off-policy policy gradient
- ☐ How to implement PG algorithms
- ☐ Actor-critic algorithms
- ☒ Batch-based actor-critic and off-policy actor-critic

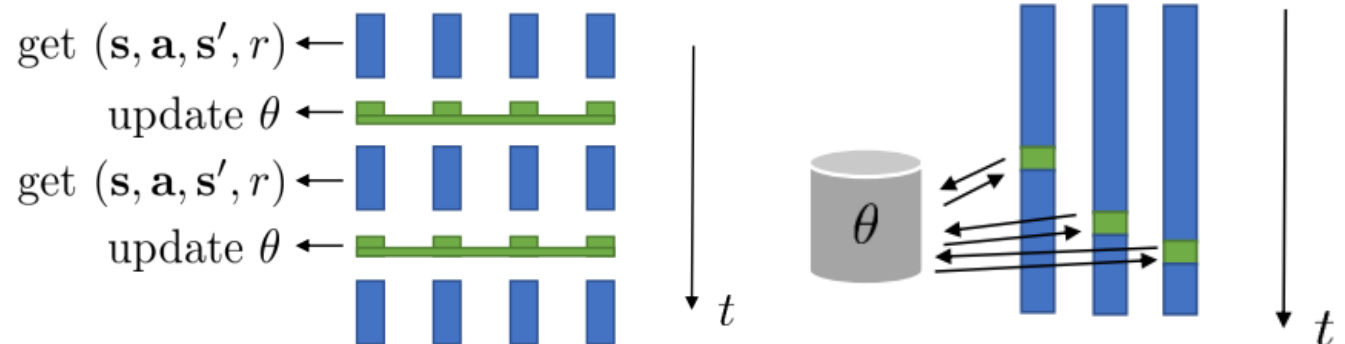
(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Batch-based Actor-Critic

online actor-critic algorithm:

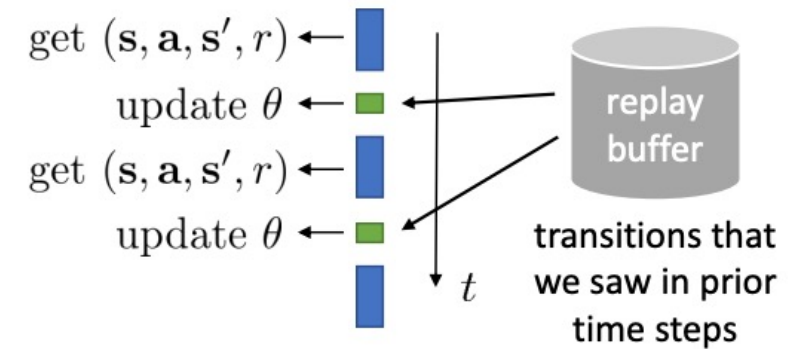
- 
1. take action $\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$
 2. update $\hat{V}_{\phi}^{\pi}$ using target $r + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}')$
 3. evaluate $\hat{A}^{\pi}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}') - \hat{V}_{\phi}^{\pi}(\mathbf{s})$
 4. $\nabla_{\theta} J(\theta) \approx \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}|\mathbf{s}) \hat{A}^{\pi}(\mathbf{s}, \mathbf{a})$
 5. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

- ❑ All DL successes rely on batch-based learning (SGD on mini batches)
- ❑ Update θ and ϕ in batches?
- ❑ Option 1: synchronized parallel actor-critic (running the policy in N environments/threads)
 - Problem: expensive!
- ❑ Option 2: asynchronous parallel actor-critic
 - Problem: (slightly) biased



Off-policy Actor-Critic

- ❑ Instead of creating many environments, can we directly sample from a replay buffer (similar to Q-learning)?
- ❑ I.e., off-policy actor-critic!
- ❑ Question: Does a naïve design (below) work?
 - No!
 - Why?



online actor-critic algorithm:

1. take action $\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$, store in $\mathcal{R}$
2. sample a batch $\{\mathbf{s}_i, \mathbf{a}_i, r_i, \mathbf{s}'_i\}$ from buffer $\mathcal{R}$
3. update $\hat{V}_{\phi}^{\pi}$ using targets $y_i = r_i + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}'_i)$ for each $\mathbf{s}_i$
4. evaluate $\hat{A}^{\pi}(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}'_i) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_i)$
5. $\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^{\pi}(\mathbf{s}_i, \mathbf{a}_i)$
6. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$



$$\mathcal{L}(\phi) = \frac{1}{N} \sum_i \left\| \hat{V}_{\phi}^{\pi}(\mathbf{s}_i) - y_i \right\|^2$$

batch size

Off-policy Actor-Critic

❑ Question: Does a naïve design (below) work?

- No!
- Why?

❑ The value estimation V_ϕ^π is wrong!

- Data in the buffer are not necessarily from the current π

❑ The policy gradient is wrong!

- Remember that policy gradient theorem is **on-policy**! I.e., transition pairs must come from the current policy.

online actor-critic algorithm:

- 
1. take action $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$, store in $\mathcal{R}$
 2. sample a batch $\{\mathbf{s}_i, \mathbf{a}_i, r_i, \mathbf{s}'_i\}$ from buffer $\mathcal{R}$
 3. update $\hat{V}_\phi^\pi$ using targets $y_i = r_i + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i)$ for each $\mathbf{s}_i$
 4. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
 5. $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
 6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



$$\mathcal{L}(\phi) = \frac{1}{N} \sum_i \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$

↑
batch size

Off-policy Actor-Critic

- ❑ How to solve this?
- ❑ Option 1: Using the previously derived off-policy policy gradient methods
 - Issue: need to keep track of all π_θ in the buffer

$$\nabla_{\theta'} J(\theta') \approx E_{(s,a) \sim \pi_\theta} \left[\frac{\pi_{\theta'}(a|s)}{\pi_\theta(a|s)} Q^{\pi_{\theta'}}(s, a) \nabla_{\theta'} \log \pi_{\theta'}(a|s) \right]$$

- ❑ Option 2 (more practical): use some “tricks” in following slides
 - Used in many actor-critic algorithms such as SAC (soft actor-critic, 2018)

online actor-critic algorithm:

- 
1. take action $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$, store in $\mathcal{R}$
 2. sample a batch $\{\mathbf{s}_i, \mathbf{a}_i, r_i, \mathbf{s}'_i\}$ from buffer $\mathcal{R}$
 3. update $\hat{V}_\phi^\pi$ using targets $y_i = r_i + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i)$ for each $\mathbf{s}_i$
 4. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
 5. $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
 6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



$$\mathcal{L}(\phi) = \frac{1}{N} \sum_i \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$

↑
batch size

Off-policy Actor-Critic

online actor-critic algorithm:

- 
1. take action $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$, store in $\mathcal{R}$
 2. sample a batch $\{\mathbf{s}_i, \mathbf{a}_i, r_i, \mathbf{s}'_i\}$ from buffer $\mathcal{R}$
 3. update $\hat{V}_\phi^\pi$ using targets $y_i = r_i + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i)$ for each $\mathbf{s}_i$
 4. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
 5. $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
 6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



$$\mathcal{L}(\phi) = \frac{1}{N} \sum_i \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$

↗
batch size

- ❑ The first issue: $V^\pi(s)$ assumes the action at s is from π , but $Q^\pi(s, a)$ doesn't! a can be anything!
- ❑ So we can estimate the Q value instead!
- ❑ Compute target: $y_i = r_i + \gamma Q_\phi^\pi(s'_i, \mathbf{a}'_i = \pi_\theta(s'_i))$. Then update the Q value using the loss

$$\sum_i \frac{1}{2} \left(y_i - Q_\phi^\pi(s_i, a_i) \right)^2$$

- Key: $\mathbf{a}'_i$ is NOT from the buffer. $\mathbf{a}'_i$ is sampled from the policy!

Off-policy Actor-Critic

online actor-critic algorithm:

- 
1. take action $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$, store in $\mathcal{R}$
 2. sample a batch $\{\mathbf{s}_i, \mathbf{a}_i, r_i, \mathbf{s}'_i\}$ from buffer $\mathcal{R}$
 3. update $\hat{V}_\phi^\pi$ using targets $y_i = r_i + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i)$ for each $\mathbf{s}_i$
 4. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
 5. $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
 6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



$$\mathcal{L}(\phi) = \frac{1}{N} \sum_i \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$

↗
batch size

- ❑ The second issue: can not do policy gradient for (s_i, a_i) NOT from π_θ
- ❑ Solution: using the same trick: compute $\nabla_\theta \log \pi_\theta(a_i^\pi | s_i)$ where $a_i^\pi \sim \pi_\theta(s_i)$
 - Still use s_i from the buffer
 - But a_i^π is sampled from the policy!

Off-policy Actor-Critic

❑ Combining these two solutions, we have a practical algorithm:

online actor-critic algorithm:

- 
1. take action $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$, store in $\mathcal{R}$
 2. sample a batch $\{\mathbf{s}_i, \mathbf{a}_i, r_i, \mathbf{s}'_i\}$ from buffer $\mathcal{R}$
 3. update $\hat{Q}_\phi^\pi$ using targets $y_i = r_i + \gamma \hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}'_i)$ for each $\mathbf{s}_i, \mathbf{a}_i$
 4. $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i^\pi | \mathbf{s}_i) \hat{Q}^\pi(\mathbf{s}_i, \mathbf{a}_i^\pi)$ where $\mathbf{a}_i^\pi \sim \pi_\theta(\mathbf{a}|\mathbf{s}_i)$
 5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

❑ Key: $\mathbf{a}'_i$ and $\mathbf{a}_i^\pi$ are NOT from the buffer!

❑ Remaining issue: No baselines (directly using Q rather than the advantage)

- Okay in practice: Off-policy gives us large batch, and we can sample $\mathbf{a}_i^\pi$ many times

❑ Another remaining issue: this procedure does not optimize over $p_\theta(s)$ (because s_i doesn't come from π_θ), but over a “broader” distribution

- Nothing we can do here. Just accept it :)
- Okay in practice: give us a slightly less optimal but more robust policy

Off-policy Actor-Critic

□ Combining these two solutions, we have a practical algorithm:

online actor-critic algorithm:

- 
1. take action $\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$, store in $\mathcal{R}$
 2. sample a batch $\{\mathbf{s}_i, \mathbf{a}_i, r_i, \mathbf{s}'_i\}$ from buffer $\mathcal{R}$
 3. update $\hat{Q}_{\phi}^{\pi}$ using targets $y_i = r_i + \gamma \hat{Q}_{\phi}^{\pi}(\mathbf{s}'_i, \mathbf{a}'_i)$ for each $\mathbf{s}_i, \mathbf{a}_i$
 4. $\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_i^{\pi} | \mathbf{s}_i) \hat{Q}^{\pi}(\mathbf{s}_i, \mathbf{a}_i^{\pi})$ where $\mathbf{a}_i^{\pi} \sim \pi_{\theta}(\mathbf{a} | \mathbf{s}_i)$
 5. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

□ Many variants to estimate the Q value (e.g., techniques we introduced in earlier lectures)

□ A commonly used practical algorithm: SAC

**Soft Actor-Critic:
Off-Policy Maximum Entropy Deep Reinforcement
Learning with a Stochastic Actor**

SAC: Soft Actor-Critic

Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor

Tuomas Haarnoja¹ Aurick Zhou¹ Pieter Abbeel¹ Sergey Levine¹

- ❑ Off-policy actor-critic + maximum entropy principle
- ❑ Objective function:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_{\pi}} [r(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_t))]$$

- ❑ $\mathcal{H}$ is the entropy measure. The entropy maximization leads to policies that can
 - Explore more
 - Capture multiple modes of near-optimal strategies (i.e., if there exist multiple options that seem to be equally good, the policy should assign each with an equal probability to be chosen).

Unify PG and Actor-Critic

□ General advantage estimation (GAE, Schulman et al., 2016):

Policy gradient methods maximize the expected total reward by repeatedly estimating the gradient $g := \nabla_{\theta} \mathbb{E} [\sum_{t=0}^{\infty} r_t]$. There are several different related expressions for the policy gradient, which have the form

$$g = \mathbb{E} \left[\sum_{t=0}^{\infty} \Psi_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right], \quad (1)$$

where Ψ_t may be one of the following:

- | | |
|--|---|
| 1. $\sum_{t=0}^{\infty} r_t$: total reward of the trajectory. | 4. $Q^{\pi}(s_t, a_t)$: state-action value function. |
| 2. $\sum_{t'=t}^{\infty} r_{t'}$: reward following action a_t . | 5. $A^{\pi}(s_t, a_t)$: advantage function. |
| 3. $\sum_{t'=t}^{\infty} r_{t'} - b(s_t)$: baselined version of previous formula. | 6. $r_t + V^{\pi}(s_{t+1}) - V^{\pi}(s_t)$: TD residual. |

The latter formulas use the definitions

$$V^{\pi}(s_t) := \mathbb{E}_{s_{t+1:\infty}, a_{t:\infty}} \left[\sum_{l=0}^{\infty} r_{t+l} \right] \quad Q^{\pi}(s_t, a_t) := \mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty}} \left[\sum_{l=0}^{\infty} r_{t+l} \right] \quad (2)$$

$$A^{\pi}(s_t, a_t) := Q^{\pi}(s_t, a_t) - V^{\pi}(s_t), \quad (\text{Advantage function}). \quad (3)$$

Rethink DDPG

□ DDPG (deep deterministic policy gradient, Lillicrap et al, ICLR'16)

for $t = 1, T$ **do**

Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise

Execute action a_t and observe reward r_t and observe new state s_{t+1}

Store transition (s_t, a_t, r_t, s_{t+1}) in R

Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$

Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$

Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

Rethink DDPG

□ Recap: policy gradient theorem

$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [Q^{\pi}(s, a) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

□ **Deterministic policy gradient theorem:**

- This paper (DPG) proves the deterministic policy gradient theorem
- A special case of the general (stochastic) policy gradient theorem!

Deterministic Policy Gradient Algorithms

David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, Martin Riedmiller

Proceedings of the 31st International Conference on Machine Learning, PMLR 32(1):387-395, 2014.

$$J(\theta) = E_{s \sim p_{\theta}} [Q^{\pi}(s, \pi_{\theta}(s))]$$

$$\nabla_{\theta} J(\theta) = E_{s \sim p_{\theta}} [\nabla_a Q^{\pi}(s, a) \Big|_{a=\pi_{\theta}(s)} \nabla_{\theta} \pi_{\theta}(s)]$$

Thank You!